

“평균 최대화 (average)” 문제 풀이

작성자: 조승현

부분문제 1

수열의 가능한 총 상태는 2^N 개를 넘지 않는다. 각 상태를 정점으로, 들어내기 연산을 방향성 간선으로 생각하여 시작점으로부터 도달 가능한 정점을 모두 탐색하면 해결할 수 있다.

부분문제 2

$D[i][j][k]$: 초기 수열이 $A[i], \dots, A[j]$ 일 때 총 k 개의 원소가 제거된 경우 제거된 원소들의 합의 최솟값.

위와 같이 정의하면 (i, j) 에서 들어내기 연산을 하는 경우는 따로 처리하고, 그 외의 경우에는 i 와 j 사이에 남아있는 원소가 있을 것이므로 모든 $i < t < j$ 에 대해 $D[i][t]$ 와 $D[t][j]$ 로부터 업데이트해주면 $O(N^5)$ 시간에 문제를 해결할 수 있다.

부분문제 3

부분문제 2의 풀이에서 모든 $i < t < j$ 에 대해 갱신을 해줄 필요가 없고, $A[i+1], \dots, A[j-1]$ 중 최솟값이 $A[t]$ 인 t 에 대해서만 갱신해주어도 충분하다. 따라서, $O(N^4)$ 시간에 문제를 해결할 수 있다. 이 때 상수가 매우 작기 때문에 $N = 250$ 인 경우에도 빠른 시간에 동작한다.

부분문제 4

모든 $A[i]$ 가 4를 넘지 않을 때, 4만 들어내는 경우 평균이 증가할 수 없으므로 무조건 손해이고, 문제에서 주어지는 구간이 막힌 수열이기 때문에 2를 들어내려면 1만 남게 되어 평균이 증가하지 않는다. 따라서, 3과 4를 들어내는 방법들만 고려하면 충분하다. 이러한 들어내기 방법들은 서로 겹치지 않는 구간들이며, 이 유효한 구간들을 평균이 작은 순서대로 제거하는 것이 항상 최적이다.

부분문제 5

막힌 수열인 부분수열은 최대 $2N$ 개이고, 이 부분수열들은 서로 교차하지 않는다. 즉, 구간으로 표현했을 때 서로 포함관계이거나 겹치는 부분이 없다. $A[i], \dots, A[j]$ 가 막힌 수열인 구간 $[i, j]$ 들을 포함 관계에 따라 트리 구조로 표현할 수 있다. 부분문제 2, 3의 풀이에서 나왔던 다이나믹 프로그래밍을 그대로 사용하여, $D[i][j]$ 를 노드 i 에 해당하는 구간에서 j 개 지웠을 때 그 합의 최솟값으로 놓으면 다이나믹 프로그래밍으로 $O(N^2)$ 시간에 문제를 해결할 수 있다.

부분문제 6

다이나믹 프로그래밍으로 $D[i][j]$ 를 계산하는 대신에, 노드 i 에 해당하는 구간에서 x 개를 제거했을 때 합 y 로 가능한 경우를 모두 좌표평면 상의 점 (x, y) 에 나타내고, 이의 convex hull을 생각하자. 부모의 convex hull은 자식의 convex hull의 민코프스키 합이 되고, 해당 노드의 구간 자체에 들어내기 연산을 했을 때에 해당하는 점 하나를 추가하는 식으로 업데이트가 된다. convex hull로 설명을 했지만, 실제로는 lower hull만 있으면 되기 때문에 기울기 기준의 priority queue로 저장하면서 small-to-large 트릭으로 관리해주면 된다. 이를 통해 $O(N \log^2 N)$ 시간에 문제를 해결할 수 있다.

부분문제 7

$A[i] \leq 20$ 이면 tree의 depth가 20보다 작아진다. 이를 이용하면 부분문제 5의 DP를 했을 때 총 계산량이 $O(N \times 20^2)$ 를 넘지 않는다.

부분문제 8

답을 구해야 하는 구간이 여러 가지이면 priority queue나 set 내에서 구간 합 연산을 해야 한다는 문제가 발생한다. 간단하게는 balanced binary search tree를 통해 구현하면 이를 해결할 수 있다. BBST를 쓰지 않고도 세그먼트 트리를 통해 문제를 해결할 수 있는데, 처음에 priority queue를 small to large trick을 통해 계산해준 다음, priority queue에 들어가는 모든 원소를 정렬 후 리넘버링하여 다시 동일한 과정을 해주면 이제 세그먼트 트리를 이용해 $O(N \log^2 N)$ 시간에 문제를 해결할 수 있다.

“과일 게임 (fruitgame)” 문제 풀이

작성자: 이온조

부분문제 1

제한이 매우 작기 때문에 쿼리마다 수행할 수 있는 모든 방법을 백트래킹을 통해 시도해보면 $O(QN!)$ 시간에 문제를 해결할 수 있다.

부분문제 2

각 과일에 대해서 그 과일을 왼쪽 끝으로 해서 만들 수 있는 가장 큰 과일을 계산할 수 있다. 이는 스택을 사용하여 과일을 합치는 것이 가능할 때마다 합쳐 주는 그리디한 알고리즘을 사용하면 된다. 쿼리마다 모든 과일에 대해서 이를 계산해보면 $O(QN^2)$ 시간에 문제를 해결할 수 있다.

부분문제 3

모든 과일의 번호가 2 이하인 부분문제이다. 번호가 1인 과일들이 연속으로 짝수 개만큼 위치해있는 경우에는 두 개씩 짝지어서 모두 번호가 2인 과일로 만들어주어도 된다. 하지만 번호가 1인 과일들이 연속으로 홀수 개만큼 위치해있는 경우에는 그 양옆의 과일들이 합쳐질 수 없다는 사실을 관찰할 수 있다. 그러므로 번호가 1인 과일들이 연속으로 홀수 개만큼 있는 위치들을 기준으로 과일들을 최대한 합쳐 보고, 가장 큰 과일의 크기를 구하면 된다는 사실을 알 수 있다. 이는 쿼리마다 선형 시간에 계산 가능하므로 $O(NQ)$ 시간에 문제를 해결할 수 있다.

부분문제 4

동적 계획법을 사용해보자. $D[i][j] := (i\text{번째 과일을 왼쪽 끝으로 해서 번호가 } j\text{인 과일을 만들 때 그 과일의 오른쪽 끝 위치})$ 라고 정의하자. 이 값들을 쿼리마다 $O(|A_i|N)$ 시간에 계산할 수 있고, $O(|A_i|NQ)$ 시간에 문제를 해결할 수 있다.

부분문제 5

생각해보면 부분문제 3의 풀이에서 중요한 정보는 1번 과일이 홀수 개만큼 연속해 있을 때 그 연속한 묶음의 위치라는 것을 알 수 있다. 이 위치들을 알고 있으면 그 사이에 있는 과일들을 모두 합칠 수 있기 때문에 가장 큰 과일의 크기를 계산할 수 있다. 즉, 과일이 업데이트될 때마다 연속한 과일 묶음에 대한 정보를 관리해주면 답을 빠른 시간 안에 계산할 수 있다. 이는 `std::set` 등을 사용하면 빠르게 관리할 수 있으므로 $O((N + Q) \log N)$ 시간에 문제를 해결할 수 있다.

부분문제 6

부분문제 4에서 사용했던 동적 계획법을 마찬가지로 먼저 계산해두자. DP 정보를 이용해서 구간 쿼리가 주어지면 그 구간에서 만들 수 있는 가장 큰 과일의 번호를 빠르게 구하면 좋을 것이다. $[l, r]$ 구간 쿼리가 주어졌을 때 번호가 x 인 과일을 만들 수 있는지 판별하려면 $l \leq i, D[i][x] \leq r$ 인 DP 값이 존재하는지 확인하면 된다. 이 때 유효한 DP값들에 대해서 단조성이 성립한다는 사실을 관찰할 수 있다. 그러므로 이분 탐색을 이용하여 $l \leq i$ 이고 $D[i][x]$ 가 유효한 i 중 가장 작은 i 를 찾아서, $D[i][x] \leq r$ 인지 판별하면 충분하다. 쿼리마다 모든 과일 번호 x 에 대해서 이를 확인하면 만들 수 있는 가장 큰 과일의 번호를 알 수 있고, 시간복잡도는 $O((N + Q \log N)|A_i|)$ 이다.

부분문제 7

번호가 x 인 과일이 y 개 연속해서 위치해 있다면 이를 (x, y) 로 표현하자. 이제 과일들을 $(x_1, y_1), (x_2, y_2), \dots, (x_k, y_k)$ 꼴로 표현할 수 있다. 이 때 $x_{i-1} > x_i < x_{i+1}$ 인 경우를 생각해보자. 만약 y_i 가 짝수라면, 부분문제 3의 풀이에서처럼 두 개씩 짝지어서 합쳐주어도 된다는 사실을 알 수 있다. 즉, $(x_i, y_i) \rightarrow (x_i + 1, \frac{y_i}{2})$ 변환이 가능하다. 반대로 y_i 가 홀수라면 남은 과일이 하나 생기기 때문에 (x_i, y_i) 묶음을 기준으로 양쪽의 과일들은 합쳐질 수 없다. 그러므로 $(x_i, y_i) \rightarrow (x_i, y_i - 1), (\infty, 1), (x_i, y_i - 1)$ 과 같이 변환이 가능하다. 이러한 변환이 불가능할때까지 계속 변환을 반복하면, $(\infty, 1)$ 사이에 바이토닉 수열 형태로 과일들이 있는 형태가 될 것이다. 바이토닉한 형태이기 때문에 그리디하게 작은 번호의 과일부터 차례대로 합쳐 주면 가장 큰 과일의 크기를 계산할 수 있음을 알 수 있다. 또한 $(\infty, 1)$ 을 기준으로 양 옆의 과일들은 분리되어 있기 때문에 전체 수열에서 만들 수 있는 가장 큰 과일의 번호를 알 수 있다.

이제 이 바이토닉 수열의 형태를 세그먼트 트리를 사용하여 관리해보자. 세그먼트 트리의 각 노드에 수열의 형태 정보와 그 구간 안에서 만들 수 있는 가장 큰 과일의 번호를 저장하자. 만약 구간 내에 $(\infty, 1)$ 이 두 개 이상 존재한다면 그 사이에 존재하는 과일들은 독립적으로 계산해주고, 구간의 최대 답을 갱신해줄 수 있다. 그러므로 양쪽 끝의 두 바이토닉 수열 정보를 관리하면 되므로, 노드마다 $O(|A_i|)$ 크기의 정보를 스택 등의 자료구조를 사용해 관리하면 된다. 두 노드를 합칠 때는 위의 변환을 사용하고, 만약 그 과정에서 $(\infty, 1)$ 이 두 개 이상 생긴다면 마찬가지로 그 사이의 수열에서 가장 큰 과일을 답에 갱신한 뒤, 양쪽 끝의 두 수열만 계속 관리하면 된다. 시간복잡도는 $O((N + Q)|A_i| \log N)$ 이다.

“경찰과 도둑 (police)” 문제 풀이

작성자: 이종영

부분문제 1

KOI 마을이 트리 구조를 이루기 때문에 경찰의 최적의 전략은 도둑을 향해 계속 움직이는 것임을 알 수 있다. 또한 도둑의 경로는 단순 경로를 이룬다. 따라서 DFS 등의 과정을 통해 도둑의 경로가 실제로 가능한 경로인지를 확인하면 된다.

부분문제 5

경찰과 도둑의 위치 간의 경로를 생각해보자. $V_1 \leq V_2$ 이므로 도둑은 경로에서 최대한 경찰 쪽으로 이동한 후 경찰의 위치 방향이 아닌 가장 멀리 떨어진 정점까지 이동하는 것이 최적의 전략이다.

도둑이 방향을 바꾸는 정점을 구하는 것은 binary lifting을 통해 구할 수 있다. 경찰과 도둑의 LCA를 기준으로 해당 정점이 어느 방향에 있는지를 먼저 구한 후, 해당 방향의 조상-자손 경로에서 sparse table을 활용한 binary lifting을 사용하면 된다.

이 과정에서 각 정점에서 "경찰의 위치 방향이 아닌 가장 멀리 떨어진 정점"을 빠르게 구해야 한다. 이는 흔히 rerooting dp라고 부르는 방식을 사용하면 $O(N)$ 의 전처리 후 $O(1)$ 에 구할 수 있다.

시간복잡도는 $O(N \log N + Q \log N)$ 이다.

부분문제 9

$V_1 \leq V_2$ 인 경우 부분문제 5의 풀이를 적용한다. 이제 $V_1 > V_2$ 를 가정한다.

경찰과 도둑의 위치 간의 경로를 생각해보자. $V_1 > V_2$ 이므로 도둑은 경로를 따라 이동하다가 잡히는 경우가 가능하다. 도둑의 최적의 전략은 u 까지 이동한다면 경찰에 끝까지 잡히지 않을 수 있으며 u 에서 한번 더 이동해 v 까지 이동한다면 도중에 경찰에게 잡히는 경로 상의 정점들 u 와 v 에 대해, u 까지 이동한 후 경찰의 위치 방향이 아닌 가장 멀리 떨어진 정점까지 이동하는 방법 또는 v 까지 이동한 후 경찰의 위치 방향이 아닌 가장 멀리 떨어진 정점까지 이동하다가 잡히는 방법 중 하나이다.

u 와 v 를 구하는 것은 부분문제 5와 유사한 binary lifting을 통해 구할 수 있다.

시간복잡도는 $O(N \log N + Q \log N)$ 이다. Jump table 등의 테크닉을 통해 $O(N + Q \log N)$ 에도 해결하는 것이 가능하다.

“철도 2 (railroad2)” 문제 풀이

작성자: 구재현

부분문제 1

모든 x, y 에 대해서, 답이 되는 D 를 이분탐색하여 구한다. x 에서 길이 D 이상인 직통 열차만을 사용하여 y 에 도달할 수 있는지는, 모든 직통 열차간의 거리를 전처리해두면 DFS를 사용하여 구할 수 있다. 직통 열차간의 거리를 전처리하는 것은, 각 정점에서 DFS를 하면 $O(N^2)$ 에 가능하다. 전체 시간 복잡도는 $O(N^4 \log(NX))$ 이다.

부분문제 3

N 개의 정점을 가진 완전 그래프 G 를 생각해 보자. 이 그래프에서 정점 x 와 정점 y 간의 간선 가중치는, 트리 상의 x, y 간의 거리와 동일하다. 다음과 같은 사실을 증명할 수 있다.

Lemma. G 의 최대 스패닝 트리에 속하는 직통 열차만을 타도 된다.

증명: 만약 최적해의 어떠한 열차가 G 의 최대 스패닝 트리에 속하지 않는다면, 트리 상의 경로를 이루는 직통 열차로 이 열차를 대체할 수 있다. ■

Lemma. $joy(x, y)$ 는 G 의 스패닝 트리 상에서 x, y 를 잇는 경로의 간선 가중치 최솟값이다.

증명: 트리 상의 경로는 유일하기 때문에, x, y 를 잇는 유일한 경로를 따라서 직통 열차를 타는 것이 최적해 중 하나이다. 이 때의 즐거움은, 경로 상에 있던 가장 짧은 직통 열차의 길이와 동일하다. 이는 간선 가중치의 최솟값에 대응된다. ■

고로 문제는 G 의 최대 스패닝 트리 $MST(G)$ 를 구하고, 모든 정점 x, y 에 대해서 $MST(G)$ 상에서 x, y 를 잇는 경로 중 간선 가중치의 최소를 계산하면 된다. $MST(G)$ 는 프림이나 크루스칼 알고리즘으로 계산하고, 간선 가중치 최소는 x 를 고정한 후 DFS를 사용하여 계산할 수 있다. 전체 시간 복잡도는 $O(N^2 \log N)$ 정도이다.

부분문제 9

완전 그래프 G 의 가중치는 트리의 경로 길이로 이루어졌다는 점을 사용하여, G 의 최대 스패닝 트리를 더 효율적으로 계산할 수 있다. 다음과 같은 사실을 관찰하자:

- 트리의 지름을 $u - v$ 라고 할 때, G 의 최대 스패닝 트리는 간선 $u - v$ 를 무조건 포함한다.
- 모든 정점 i 에 대해서, i 에서 가장 먼 정점을 $f(i)$ 라고 하면, G 의 최대 스패닝 트리는 간선 $i - f(i)$ 를 무조건 포함한다.
- $f(i)$ 는 u, v 중 하나이다. (아닐 경우 지름이라는 가정에 모순이다.)

트리의 지름을 $O(N)$ 시간에 계산한 후, 지름 $u - v$ 를 최대 스패닝 트리에 추가하자. 모든 $i \neq \{u, v\}$ 에 대해서, $f(i)$ 가 u, v 중 무엇인지를 거리를 비교하여 결정하고 (u, v 에서 DFS로 거리를 전처리할 수 있다.), $i - f(i)$ 를 최대 스패닝 트리에 추가하자. 지금까지 넣은 간선들은, 스패닝 트리에 무조건 들어가야만 하는 간선들이다. 또한, 정확히 $N - 1$ 개의 간선을 넣었기 때문에, 이렇게 찾은 트리가 정확히 최대 스패닝 트리가 된다.

계산한 $MST(G)$ 를 사용하여 $joy(x, y)$ 의 합을 계산할 수 있다. $MST(G)$ 의 간선을 가중치가 감소하는 순으로 정렬한 후, Union-Find를 사용하여 처리한 간선의 양 끝을 하나의 컴포넌트로 합쳐주자. $MST(G)$ 의 간선 $e = (x, y)$ 를 처리하는 시점에, x 의 컴포넌트에 속하는 임의의 정점 v_x 와 y 의 컴포넌트에 속하는 임의의 정점 v_y 에 대해 $joy(v_x, v_y)$ 는 e 의 가중치와 동일하다. v_x, v_y 를 잇는 경로 상의 간선 중 마지막으로 처리된 간선이 e 인데, 마지막으로 처리되었다는 것은 가중치가 가장 작다는 것이기 때문이다. 고로, 간선 e 가 $joy(x, y)$ 에 기여하는 정도는, $(e \text{의 가중치}) \times (x \text{의 컴포넌트 크기}) \times (y \text{의 컴포넌트 크기})$ 이다. 이 과정에서 모든 서로 다른 정점 쌍 x, y 가 정확히 한 번씩 고려되므로, 위 기여되는 정도를 합하면 $joy(x, y)$ 의 합을 얻을 수 있다.

전체 시간 복잡도는 MST 의 간선을 정렬하는 데 필요한 $O(N \log N)$ 이다.